

Agenda

- Arquitectura Cliente - Servidor
- HTTP
- Arquitectura Web
- API
- API REST
- Recursos Web



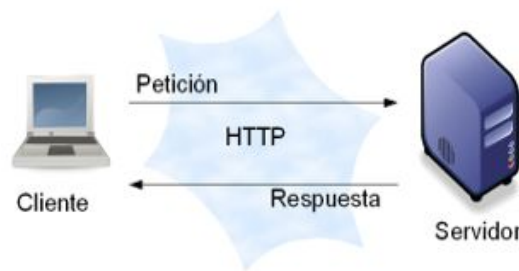


Arquitectura Cliente - Servidor

Cliente - Servidor

Participan 2 componentes:

- Un servidor que provee uno o más servicios a través de una interfaz.
- Un cliente que usa esos servicios como parte de su operación en el acceso al servidor.





Cliente - Servidor - Clasificación según responsabilidades

Una arquitectura Cliente – Servidor se puede clasificar, dependiendo las responsabilidades asignadas a sus componentes, en:

- **Cliente Activo, Servidor Pasivo**: el cliente es quien posee la mayor lógica de negocio. El servidor limita su funcionalidad a la persistencia.
 - **Cliente Pasivo, Servidor Pasivo**: ambos componentes poseen baja lógica de negocio o simplemente son considerados “componentes intermedios” de algo “más grande”.
 - **Cliente Pasivo, Servidor Activo (“Cliente liviano”)**: el servidor posee la mayor lógica de negocio; mientras que el cliente se limita a presentar los datos.
 - **Cliente Activo, Servidor Activo (“Cliente pesado”)**: la lógica de negocio está distribuida en ambos componentes. El cliente posee la lógica de presentación de los datos.
- 



Cliente - Servidor

Analizando ventajas y desventajas sobre un **Cliente Pasivo – Servidor Activo**

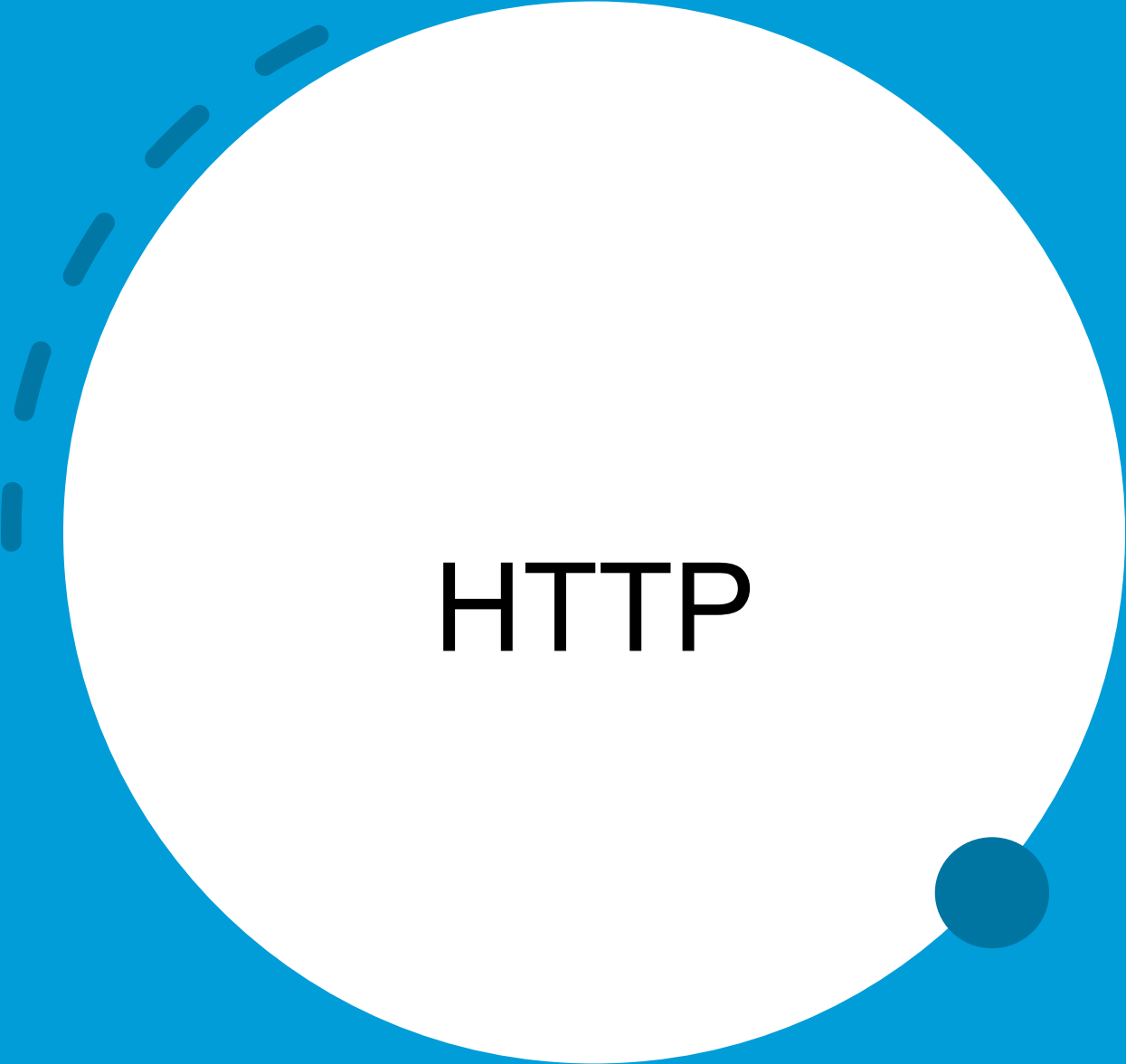
Ventajas

- **Mantenibilidad:** Cambios de funcionalidad Centralizados
- **Seguridad:** Centralización de Control de Accesos a recursos

Desventajas (*)

- **Eficiencia (tiempo de respuesta):** El servidor puede ser un cuello de botella
- **Disponibilidad:** Único punto de falla

(*) Considerando un único Servidor

A large white circle is centered on a solid blue background. A dashed dark blue line forms an arc on the left side of the white circle. A small solid dark blue circle is positioned on the right edge of the white circle.

HTTP



HTTP

HTTP (HyperText Transfer Protocol) es un protocolo de comunicación basado en el modelo Cliente-Servidor que define cómo se deben intercambiar datos en la web.

Es un protocolo de aplicación dentro de la pila de protocolos de Internet y funciona sobre TCP (generalmente en el puerto 80 para HTTP y 443 para HTTPS).





HTTP - Funcionamiento básico

1. Cuando un cliente (normalmente un navegador) necesita acceder a un recurso en la web, envía una solicitud HTTP (HTTP request) a un servidor web.
2. Este servidor procesa la solicitud y responde con una respuesta HTTP (HTTP response) que contiene los datos solicitados o un código de estado que indica el resultado de la solicitud.





HTTP

Resumen

- Protocolo de Transferencia de Hipertexto
- Capa de Aplicación (Capa 7 - Modelo OSI)
- Sincrónico – Sin Estado
- Cliente / Servidor
- Solicitud / Respuesta
- Puerto 80 / 443 (HTTPS)

Métodos/Verbos Principales

- GET
- POST
- PUT
- DELETE
- Patch

RFC-216 (<https://tools.ietf.org/html/rfc2616>)





HTTP vs HTTPS

- **HTTP** transmite datos en texto plano.
- **HTTPS** (HTTP Secure) utiliza TLS/SSL para cifrar los datos, proporcionando confidencialidad e integridad en la comunicación.





HTTP

GET → Solicita un recurso sin modificarlo. (Ejemplo: cargar una página web)

POST → Envía datos al servidor para que sean procesados. (Ejemplo: enviar un formulario)

PUT → Actualiza o crea un recurso en el servidor.

DELETE → Elimina un recurso del servidor.

PATCH → Modifica parcialmente un recurso.



HTTP - Códigos de Estado





HTTP - Códigos de Estado

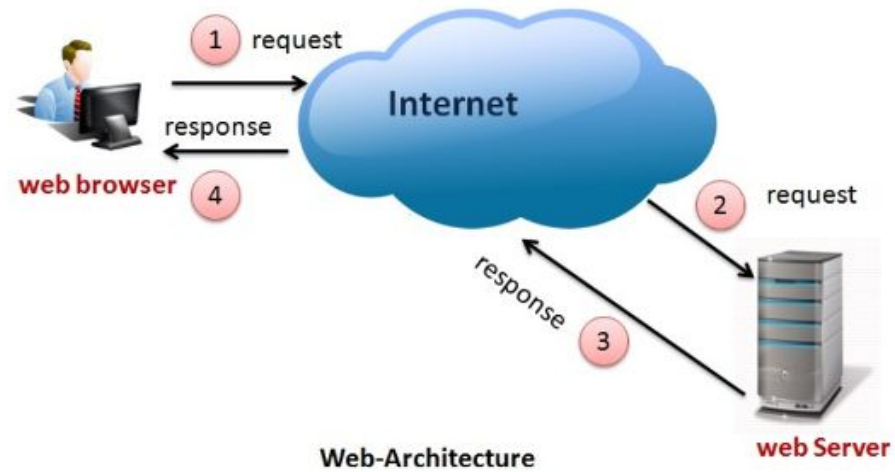
- **1xx**: Respuestas informativas. Indica que la petición ha sido recibida y se está procesando.
- **2xx**: Respuestas correctas. Indica que la petición ha sido procesada correctamente.
- **3xx**: Respuestas de redirección. Indica que el cliente necesita realizar más acciones para finalizar la petición.
- **4xx**: Errores causados por el cliente. Indica que ha habido un error en el procesamiento de la petición a causa de que el cliente ha hecho algo mal.
- **5xx**: Errores causados por el servidor. Indica que ha habido un error en el procesamiento de la petición a causa de un fallo en el servidor.



A large white circle is centered on a blue background. A dashed blue arc is positioned on the left side of the circle, and a solid blue dot is on the right side.

Arquitectura Web

Arquitectura Web

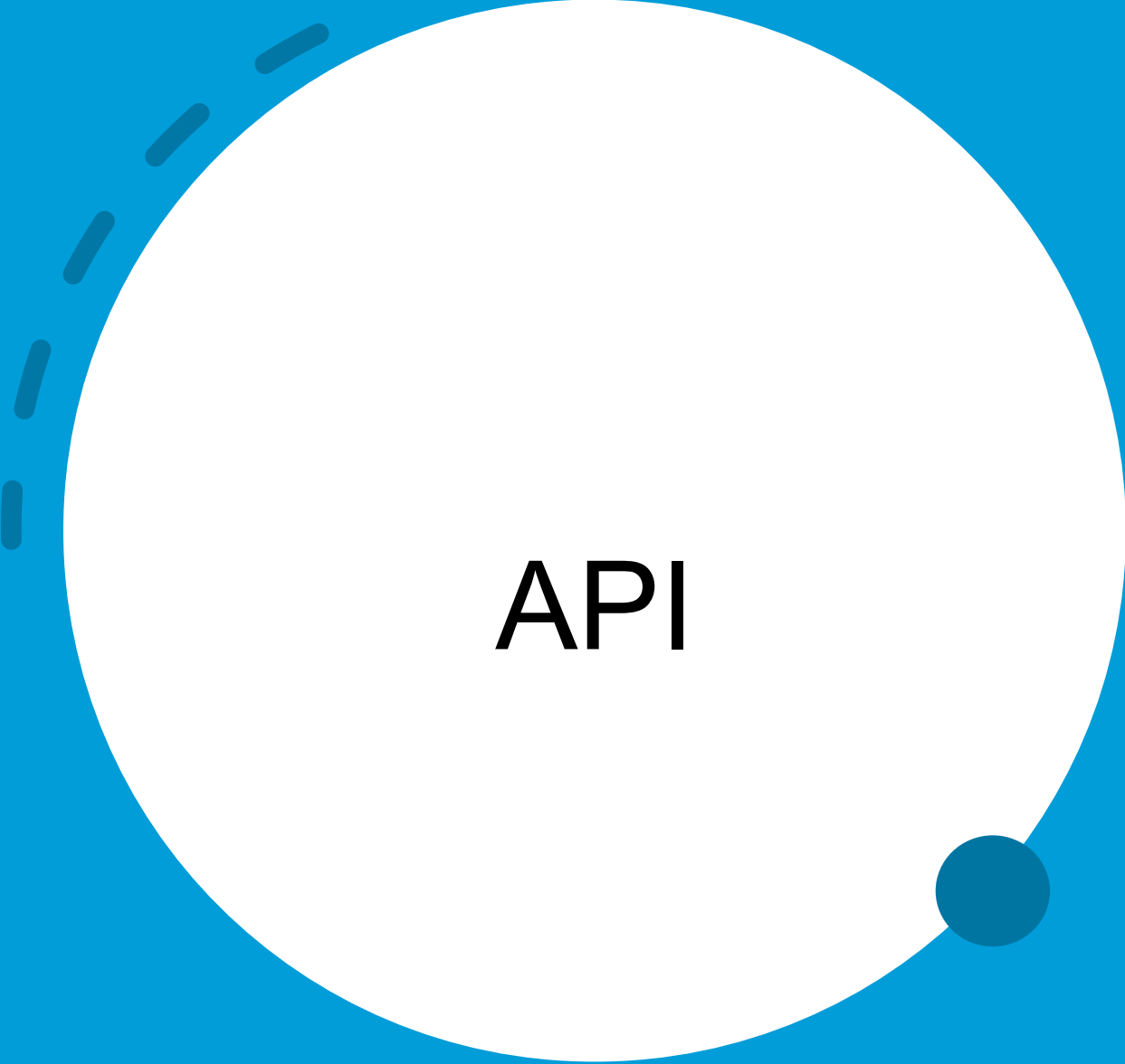




Arquitectura Web

- La Arquitectura Web es el conjunto de principios, tecnologías y estructuras que definen cómo se organizan y comunican los componentes de una aplicación web.
- Se basa en la interacción entre clientes y servidores a través de protocolos como HTTP/HTTPS, permitiendo la distribución de información y la ejecución de aplicaciones en la web.



A large white circle is centered on a solid blue background. A dashed dark blue line arcs from the top-left towards the circle. A small solid dark blue circle is positioned on the right edge of the white circle.

API



API

- Una ***interfaz de programación de aplicaciones*** (API) es un conjunto de herramientas, definiciones y protocolos que se usan para diseñar e integrar aplicaciones.
- Permite que un producto o servicio se comuniquen con otros productos y servicios, sin la necesidad de saber cómo se implementan internamente.



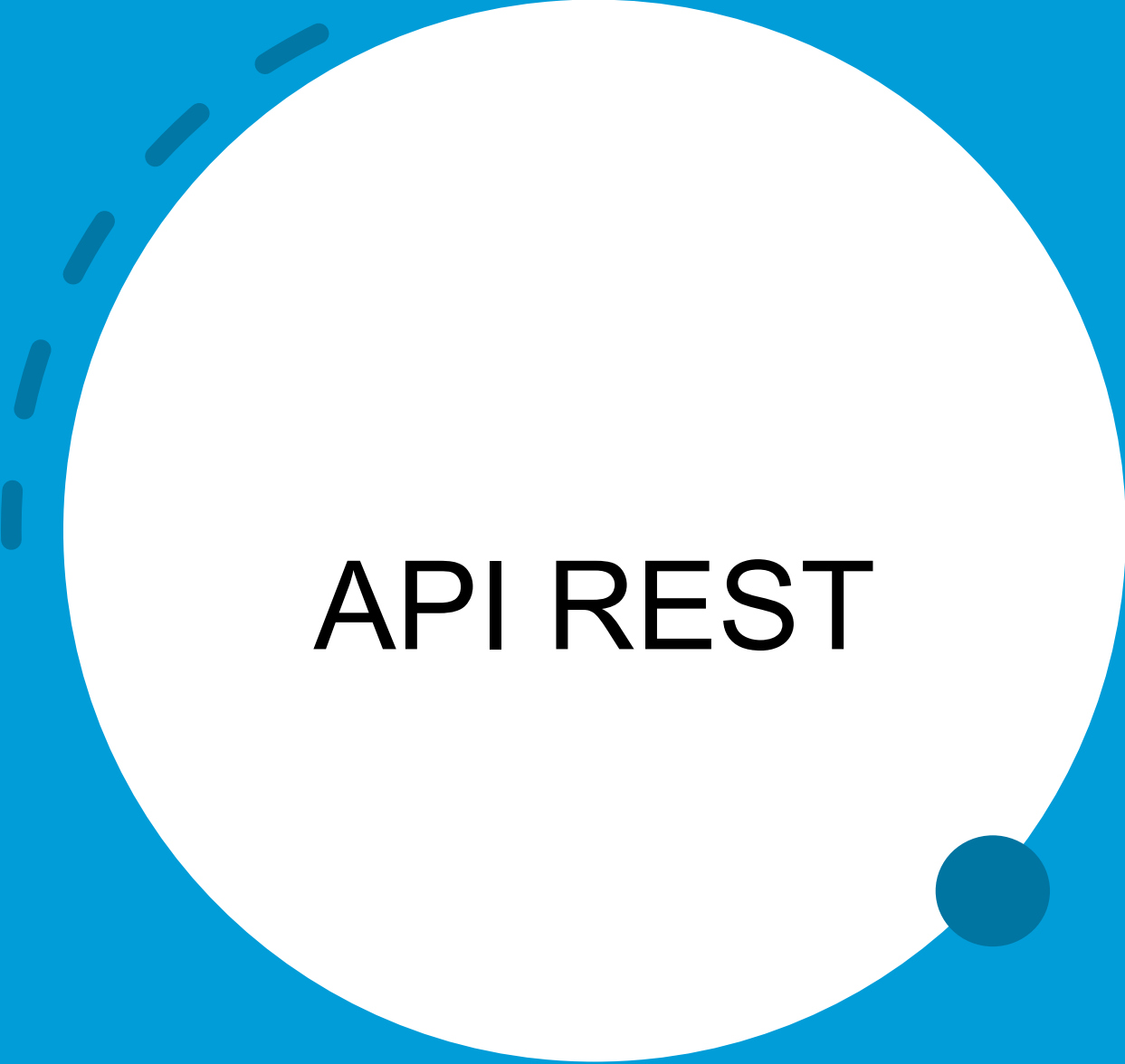


API

Por ejemplo, APIs podrían ser:

- API **Stream** de Java, que brinda un conjunto de métodos para poder manipular un Stream de Datos.
- **JDBC** (Java Database Connectivity), proporciona clases e interfaces para interactuar con bases de datos relacionales.
- **Win32 API** (Windows), proporciona funciones y estructuras de datos para interactuar con los recursos del Sistema, como ventanas, archivos, procesos, memoria, I/O, etc.
- **POSIX API** (Linux), proporciona funciones para realizar operaciones de bajo nivel, como la gestión de archivos, manejo de procesos, concurrencia, etc.





API REST

API REST

REST o Representational State Transfer es un enfoque de diseño de comunicación de Arquitectura a la hora de realizar una comunicación entre cliente y servidor.





API REST

- Se apoya sobre el protocolo HTTP.
- Crea una solicitud HTTP que contiene toda la información necesaria, es decir, una REQUEST a un servidor tiene toda la información necesaria y solo espera una RESPONSE, ósea una respuesta en concreto.
- Se apoya en los métodos básicos de HTTP, como son:
 - Post
 - Get
 - Put
 - Patch
 - Delete





API REST

- Todos los objetos se manipulan mediante URI (identificador uniforme de recursos)
- Utiliza como formato de intercambio **JSON** / XML
 - JSON es un formato ligero de intercambio de datos.
 - Leerlo y escribirlo es simple para humanos, mientras que para las máquinas es simple interpretarlo y generarlo.





API REST - Ejemplo de json

```
{  
  "id": 2,  
  "email": "janet.weaver@reqres.in",  
  "first_name": "Janet",  
  "last_name": "Weaver",  
  "avatar": "https://reqres.in/img/faces/2-image.jpg"  
}
```





API REST - Ejemplo de xml

```
<User>
  <id>2</id>
  <email>janet.weaver@reqres.in</email>
  <first_name>Janet</first_name>
  <last_name>Weaver</last_name>
  <avatar>https://reqres.in/img/faces/2-image.jpg</avatar>
</User>
```





API REST

Entonces... ¿cómo generamos las rutas del Servidor siguiendo el enfoque REST?





Recursos Web



Recursos Web

- Cuando diseñamos un Sistema con arquitectura Web debemos pensar en qué recursos vamos a permitir manipular.
- Podemos decir que un Recurso es una entidad que puede ser manipulada: puede ser dada de alta, modificada, eliminada o listada (buscada).
- Para poder identificar qué recursos debe exponer un Sistema se deben tener en cuenta los casos de uso.





Recursos Web

Si consideramos que nuestro servidor permite la manipulación del recurso “**User**”, algunas rutas podrían ser:

- GET /users
- GET /users/1
- POST /users
- PUT /users/1
- DELETE /users/1





Recursos Web

- GET /users -> Se obtienen todos los usuarios
- GET /users/1 -> Se obtiene el usuario con id 1
- POST /users -> Se genera un nuevo usuario (se deben enviar los datos del nuevo recurso por el Body de la Request)
- PUT /users/1 -> Se actualiza el usuario con id 1 (se deben enviar los datos del nuevo recurso por el Body de la Request)
- DELETE /users/1 -> Se borra el usuario con id 1





Recursos Web

- GET /users - 200 OK
- GET /users/1 - 200 OK
- POST /users - 201 Created
- PUT /users/1 - 200 OK
- DELETE /users/1 - 204 No Content





Recursos Web

- Siguiendo REST, las rutas siempre deberían llevar nombres de recursos.
- Siguiendo con lo anterior, no deberíamos utilizar verbos como nombres de rutas ya que indicarían acciones (para eso existen los verbos HTTP).
- Deberíamos utilizar los códigos de estado HTTP para indicar cómo finalizó la operación.



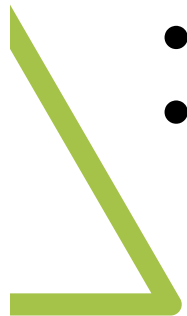
A decorative graphic consisting of a dashed blue arc on the left side of the white circle and a solid dark blue dot on the right side of the white circle.

REST Avanzado



Uso de las URL

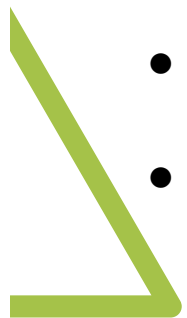
- Existen 3 partes en una URL **dominio** / **base** / **query_string**
 - GET **http://starbucks.com/drinks/34**
 - GET **http://amazon.com/books/?date_gt=2010&price_lt=100**
 - POST **http://empresaX.com/api/compra**
-
- Dominio: no forma parte de HTTP, se usa para enrutar entre nodos
 - Base: es la parte obligatoria para hacer referencia a un recurso o listado
 - Query String: Key/value opcional, en general se usa para hacer filtros





Jerarquía

- GET <http://starbucks.com/drinks/34/ingredients>
- GET <http://starbucks.com/drinks/34/ingredients/33>
- POST <http://empresaX/api/compra/65/items>
- En general las jerarquías de más “arriba” muestran la descripción completa del recurso, pero si se relaciona con otro recurso, se muestra una versión “reducida del mismo” (un nombre / descripción / id / cantidad / etc)
- Depende del diseño de datos de la API y que casos de uso considera más importantes
- A veces, cuando se necesitan datos específicos de una lista larga de recursos, que pertenecen a un recurso base, se necesitan hacer tantos requests como datos.
- Fuera ya del alcance de la materia, esto se puede atacar utilizando GraphQL, la cual permite al cliente realizar una consulta específica. Como contraparte el mismo debe conocer el esquema de datos de la API (no de la base)





Listados

GET /products *¿Qué pasa cuando el listado es GRANDE?*

Paginación

- Página / tamaño de página / orden
- Se suele expresar como query string: ?page=2&psize=50
- Tiene valores default y limites: por ejemplo page=1 si no hay querystring y psize < 1000
- Respuesta es una página:



```
{ data: [{id:1, text:"algo"}, {id:2, text:"xxx"}, ...],  
  page:"1",  
  pageSize:"50",  
  Total:"500" }
```



Procesos

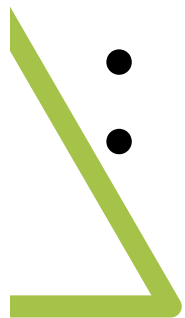
- ¿Qué pasa cuando quiero realizar un proceso sobre un recurso, que no es un ABM? Por ejemplo "recategorizar empresa", "validar licitación"

```
/licitacion/200/validar?
```

```
/licitacion/200?validar=True ...
```

Si es suficientemente importante se puede crear un recurso que represente el proceso en sí, por ejemplo

- POST /validacion -> enviamos el id de la licitación → retorna un ID
- GET /validacion/333 -> si terminó, nos retorna los datos, sino nos puede retornar un mensaje de en proceso



Idempotencia



HTTP Method	Safe	Idempotent
GET	✓	✓
POST	✗	✗
PUT	✗	✓
DELETE	✗	✓
OPTIONS	✓	✓
HEAD	✓	✓

Es un principio que indica que realizar una misma solicitud varias veces sobre un recurso no cambia el estado del Sistema.





¡Gracias!